Branch and Bound

이원형 교수
Dr. Lee, WonHyong
whlee@handong.edu

Branch and Bound

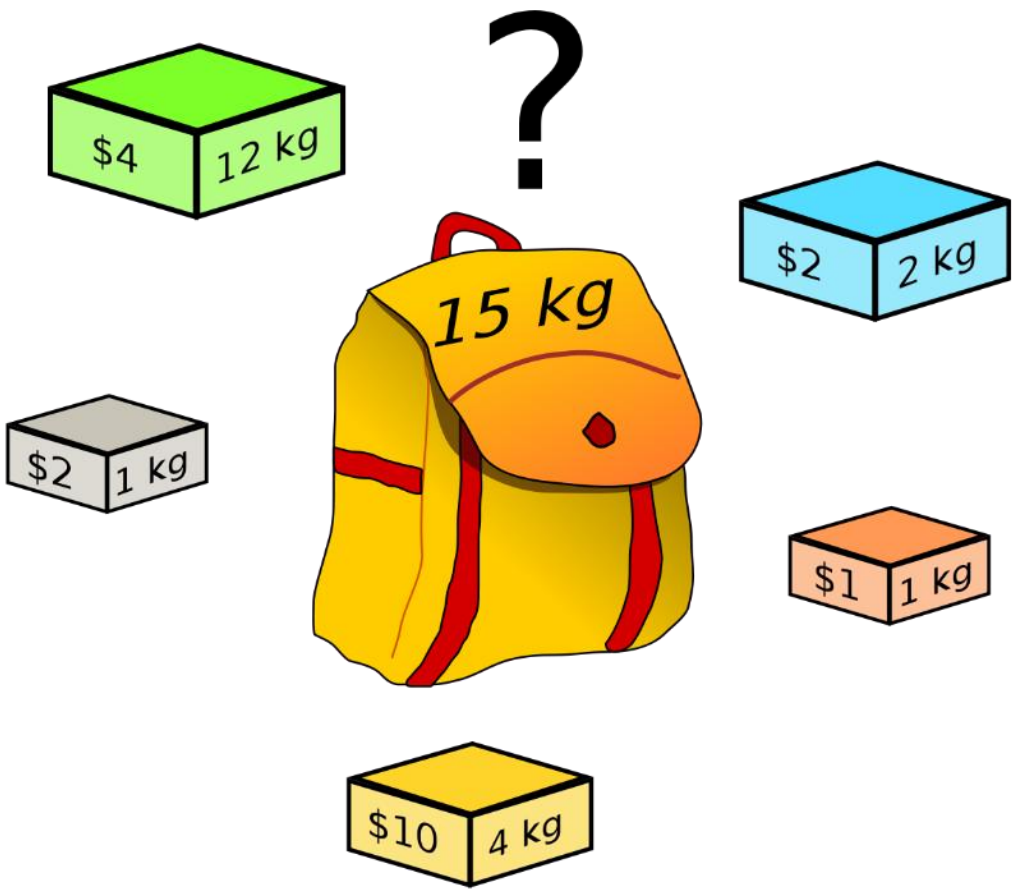
Knapsack Problem

THE KNAPSACK PROBLEM

- **The famous knapsack problem:**
  - ✓ A thief breaks into a museum. Fabulous paintings, sculptures, and jewels are everywhere.

THE KNAPSACK PROBLEM

- **The famous knapsack problem:**
  - ✓ But the thief has only brought a single knapsack.
  - ✓ What items should the thief take <u>to maximize the value</u> of packed objects?



The Knapsack Problem

- **There are two versions of the problem:**
  1. "**Fractional** knapsack problem"
     - Items are divisible: you can take any fraction of an item.
  2. "**0-1** knapsack problem" and
     - Items are indivisible: you either take an item or not.

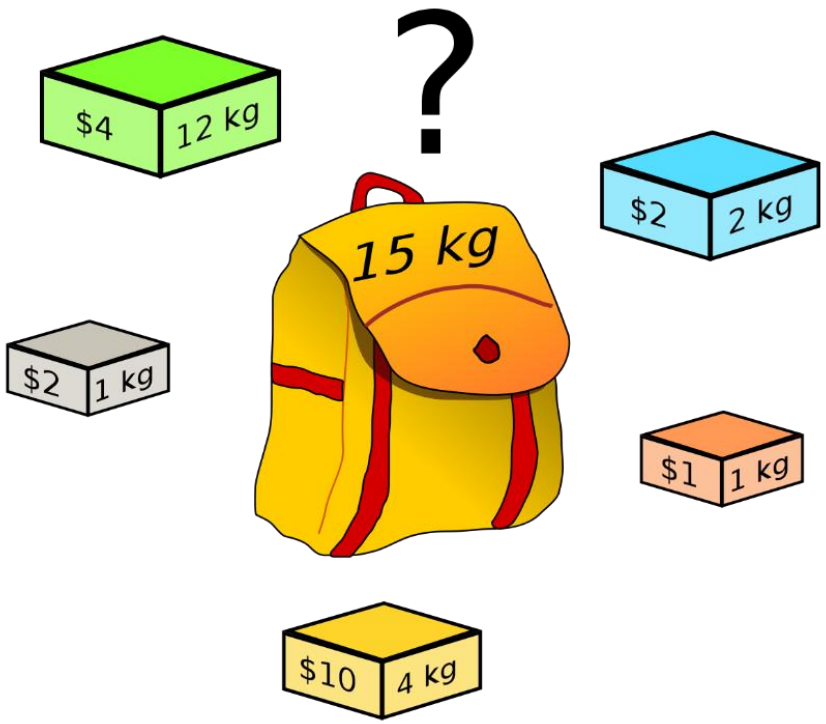
FRACTIONAL KNAPSACK PROBLEM

- To solve the fractional problem,
  - ✓ we first compute the **value per weight** (v_i/w_i) for each item.
- Following a greedy strategy,
  - ✓ the thief begins by taking as much as possible of the item with the greatest value per pound.
- If the supply of that item is exhausted and he can still carry more,
  - ✓ he takes as much as possible of the item with the next greatest value per weight, and so forth until he can't carry any more.

※ It is like the coin change example $O(n)$.

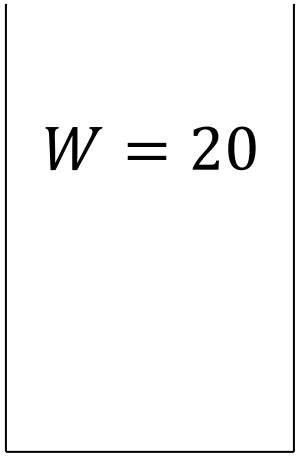
0-1 Knapsack Problem

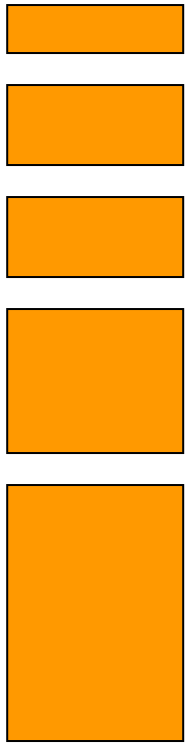
- A knapsack with maximum capacity W, and a set S consisting of n items.
- Each item i has some weight w_i and benefit value b_i.
- Suppose that all w_i, b_i, and W are integer values.



0-1 Knapsack Problem

This is a knapsack
Max weight: $W = 20$





Items	Weight w_i	Benefit value b_i
	2	3
	3	4
	4	5
	5	8
	9	10

0-1 Knapsack Problem

1. Brute force approach

✓ Let's first solve this problem with a straightforward algorithm.

- Since there are n items, there are 2^n possible combinations of items.
- We go through all combinations and find the one with the most total value and with total weight less or equal to W.
- Running time will be $O(2^n)$.

0-1 Knapsack Problem

2. **Greedy approach does not work for this problem.**
   - ✓ (although it works for fractional knapsack problem.)

3. **Dynamic programming**
   - ✓ We need to carefully identify the subproblems.

   - ✓ Let's try this:
     - If items are labeled $1 \ldots n$, then
       a subproblem would be to find an optimal solution for $S_k = \{\text{items } 1, 2, \ldots, k\}$

     - Question : Can we describe the final solution (S_n) in terms of subproblems (S_k)?
     - Answer : Unfortunately, we can **not** do that. Explanation follows….

0-1 Knapsack Problem

$w_1 = 2$ $b_1 = 3$	$w_2 = 4$ $b_2 = 5$	$w_3 = 5$ $b_3 = 8$	$w_4 = 3$ $b_4 = 4$	

Max weight: W = 20

For S_4:

Total weight: 14

total value: 20

$w_1 = 2$ $b_1 = 3$	$w_3 = 4$ $b_3 = 5$	$w_4 = 5$ $b_4 = 8$	$w_5 = 9$ $b_5 = 10$	

For S_5:

Total weight: 20

total value: 26

Item #	Weight w_i	Benefit b_i
1	2	3
2	3	4
3	4	5
4	5	8
5	9	10

S_4

S_5

Solution for S_4 is not part of the solution for S_5!!!

DEFINING A PROBLEM

- As we have seen, the solution for S_4 is not part of the solution for S_5.
- So, our definition of a subproblem S_k can not be directly used.
- Let's define another notation $B[k, W]$ to compute.
  - ✓ The benefit of the best subset of S_k that has the total weight W.

Recursive Formula for subproblems

$$B[k,W] = \begin{cases} B[k-1,W] & \text{if } w_k > W \\ \max(B[k-1,W], B[k-1,W-w_k] + b_k) & \text{otherwise} \end{cases}$$

- The benefit of the best subset of S_k that has the total weight W either contains item k or not.
- First case: $w_k > W$.
  - ✓ Item k can't be a part of the solution, since if it was, the total weight would be greater than W, which is unacceptable.
- Second case: $w_k \leq W$.
  - ✓ Then the item k <u>may</u> be in the solution, and we choose the case with greater value.

1. the best subset of S_{k-1} that has total weight W, or
2. the best subset of S_{k-1} that has total weight $W - w_k$

PSEUDOCODE

```
for w = 0 to W
  B[0,w] = 0
for i = 1 to n
  B[i,0] = 0
  for w = 1 to W
    if wi <= w // item i may be part of the solution
      if bi + B[i-1,w-wi] > B[i-1,w]
        B[i,w] = bi + B[i-1,w- wi]
      else  B[i,w] = B[i-1,w]
    else B[i,w] = B[i-1,w]  // wi > w
```

EXAMPLE

- Let's run our algorithm on the following data:
  - ✓ $n = 4$ (# of elements)
  - ✓ $W = 5$ (max weight)
  - ✓ Elements (weight, benefit value): (2,3), (3,4), (4,5), (5,6)

EXAMPLE

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0				
2	0				
3	0				
4	0				
5	0				

```
for w = 0 to W
  B[0,w] = 0
for i = 1 to n
  B[i,0] = 0
```

EXAMPLE

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0 →	0			
2	0				
3	0				
4	0				
5	0				

$i = 1$
$w_i = 2$
$b_i = 3$
$W = 1$
$W < w_i$

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

```
if wi <= w // item i may be part of the solution
  if bi + B[i-1,w-wi] > B[i-1,w]
    B[i,w] = bi + B[i-1,w- wi]
  else  B[i,w] = B[i-1,w]
else B[i,w] = B[i-1,w]  // wi > w
```

EXAMPLE

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0	0			
2	0	**3**			
3	0				
4	0				
5	0				

$i = 1$
$w_i = 2$
$b_i = 3$
$W = 2$
$W = w_i$

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

```
if wi <= w // item i may be part of the solution
  if bi + B[i-1,w-wi] > B[i-1,w]
    B[i,w] = bi + B[i-1,w- wi]
  else  B[i,w] = B[i-1,w]
else B[i,w] = B[i-1,w]  // wi > w
```

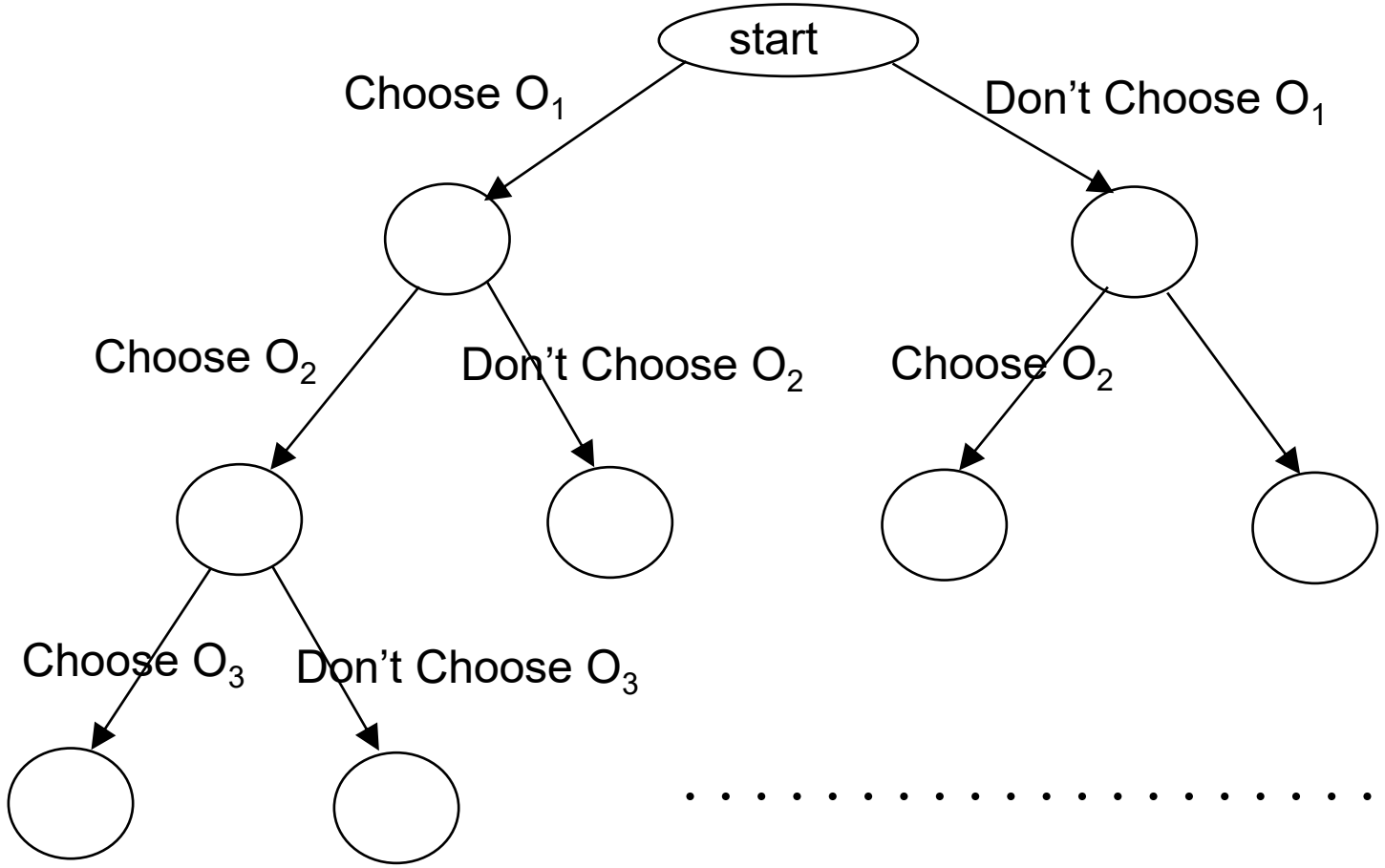
PSEUDOCODE

```
for w = 0 to W
  B[0,w] = 0
for i = 1 to n
  B[i,0] = 0
  for w = 1 to W
    if wi <= w // item i may be part of the solution
      if bi + B[i-1,w-wi] > B[i-1,w]
        B[i,w] = bi + B[i-1,w- wi]
      else  B[i,w] = B[i-1,w]
    else B[i,w] = B[i-1,w]  // wi > w
```

Time complexity: $O(n \cdot W)$

0-1 Knapsack Problem

4. **Branch and Bound** (It is <u>another design paradigm</u>!)
   - ✓ Best First Search
   - ✓ We do not visit every node but determine if it is worth or not.



Branch and Bound

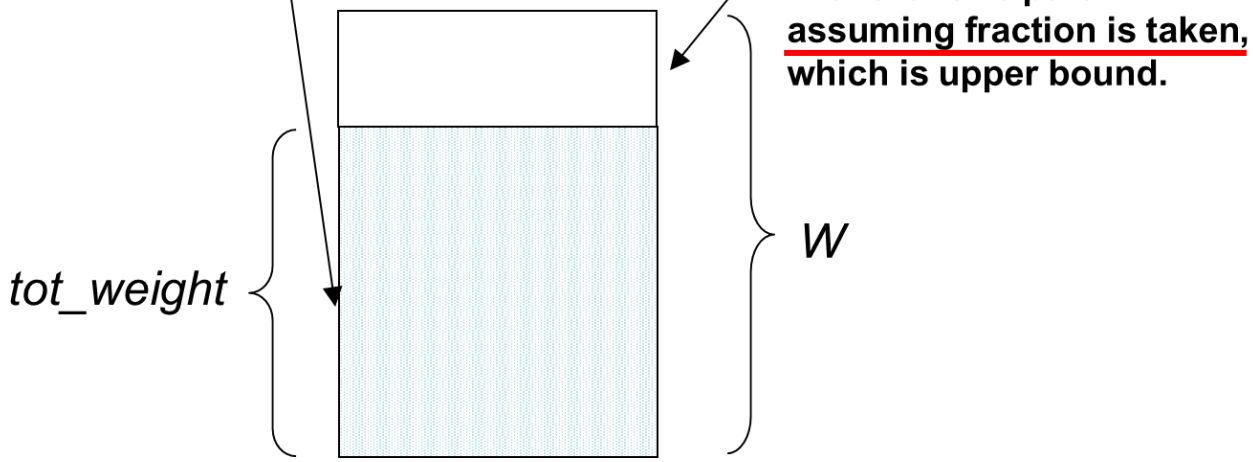
- At each node we compute three local values.
  - ✓ $weight$: sum of weights up to this node
  - ✓ $benefit$: sum of benefit value up to this node
  - ✓ $bound$: upper bound on **the benefit we could achieve** by expanding beyond this node

The **bound** is calculated by taking the items currently in the knapsack and adding the maximum possible value of the remaining items, assuming they can be taken fractionally. Although this is a theoretical limit—since items cannot actually be divided in a 0/1 Knapsack problem—it serves as a definitive **upper bound** that the actual total benefit value can never physically exceed.

- Computing '$bound$'
  - ✓ $\boldsymbol{bound}$ = benefit from tot_weight + virtual benefit from the empty place
  - ✓ At level i, benefit from tot_weight is initially $benefit$, then add b_{i+1}, b_{i+2}, … b_{k-1} to $benefit$
    - The items considered are the same as tot_weight.

$$tot_weight = weight + \sum_{j=i+1}^{k-1} w_j$$

$$\boxed{bound} = \underbrace{benefit + \sum_{j=i+1}^{k-1} b_j}\ + \underbrace{(W - tot_weight) \times \frac{b_k}{w_k}}$$



Branch and Bound

- And we need one more global variable '$max_benefit$'
  - ✓ benefit in the best solution found so far.
  - ✓ Initially, $max_benefit = 0$.
  - ✓ Afterwards at each node, if $weight < W$,

$$max_benefit = \max(max_benefit, benefit)$$

Branch and Bound

- A **node** is evaluated as **promising** or **non-promising**.
  - ✓ promising : We will expand beyond this node.
  - ✓ nonpromising : We will stop here.
- A **node** (or a vertex) is **non-promising**
  - ✓ if $bound \leq max_benefit$
    - – we can't get the benefit more than $max_benefit$ even if we expand the nodes.
  - ✓ or $weight > W$
- A **node** (or a vertex) is **promising**
  - ✓ if $bound > max_benefit$ and $weight \leq W$

Branch and Bound

- Basically, apply Best-First-Search and choose promising and unexpanded node with greatest bound.

Each node has

$$\begin{matrix} benefit \\ weight \\ bound \end{matrix}$$

Example) 4 items, $W = 16$

i	b_i	w_i	b_i/w_i
1	\$40	2	\$20
2	\$30	5	\$6
3	\$50	10	\$5
4	\$10	5	\$2

Decreasing order of b_i/w_i

Branch and Bound

$$tot_weight = weight + \sum_{j=i+1}^{k-1} w_j$$

Vertex (0,0)

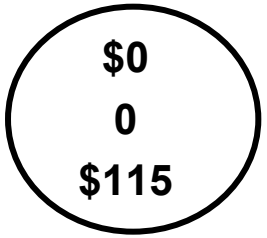
$benefit = 0, weight = 0 \leq W = 16$

$max_benefit = \$0$

$tot_weight = 0 + (2 + 5) = 7 \leq W = 16$

$bound = \$0 + (\$40 + \$30) + (16 - 7) * \$50/10 = \$115 > max_benefit$

(0,0)



b_i	w_i
\$40	2
\$30	5
\$50	10
\$10	5

$$tot_weight = weight + \sum_{j=i+1}^{k-1} w_j$$

$$bound = benefit + \sum_{j=i+1}^{k-1} b_j + (W - tot_weight) \times \frac{b_k}{w_k}$$

Vertex (1,1)

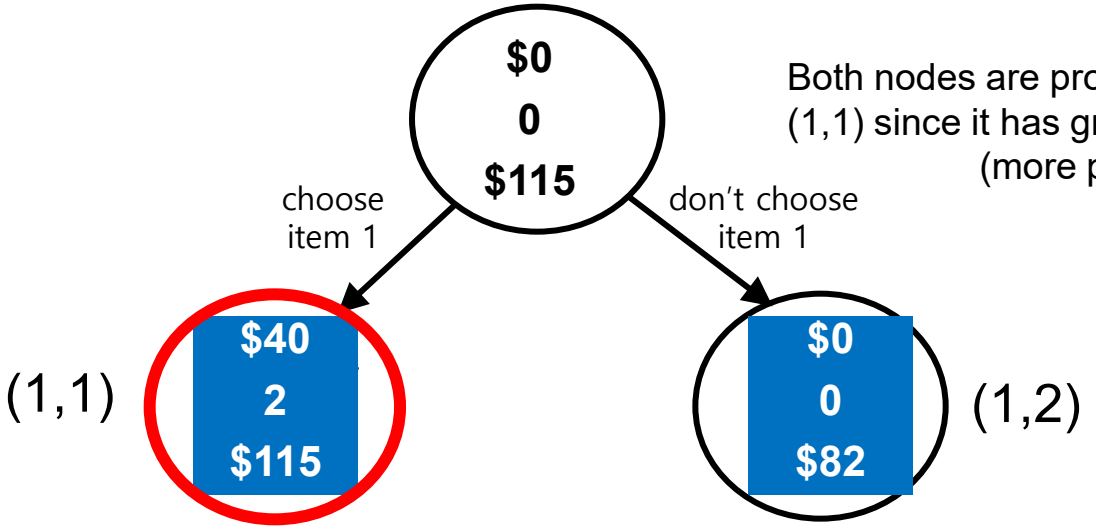
$benefit$ = 0+\$40 = \$40

$weight$ = 0+2 = 2 $\leq W = 16$

$max_benefit$ = max(\$0, \$40) = \$40

tot_weight = 2+ (5) = 7 $\leq W = 16$

$bound$ = \$40+(\$30)+(16-7)*\$50/10=\$115 $>$ $max_benefit$



Both nodes are promising, but choose (1,1) since it has greatest bound (more promising)

b_i	w_i
\$40	2
\$30	5
\$50	10
\$10	5

$$tot_weight = weight + \sum_{j=i+1}^{k-1} w_j$$

$$bound = benefit + \sum_{j=i+1}^{k-1} b_j + (W - tot_weight) \times \frac{b_k}{w_k}$$

Vertex (2,1)

$benefit$ = \$40+\$30 = \$70

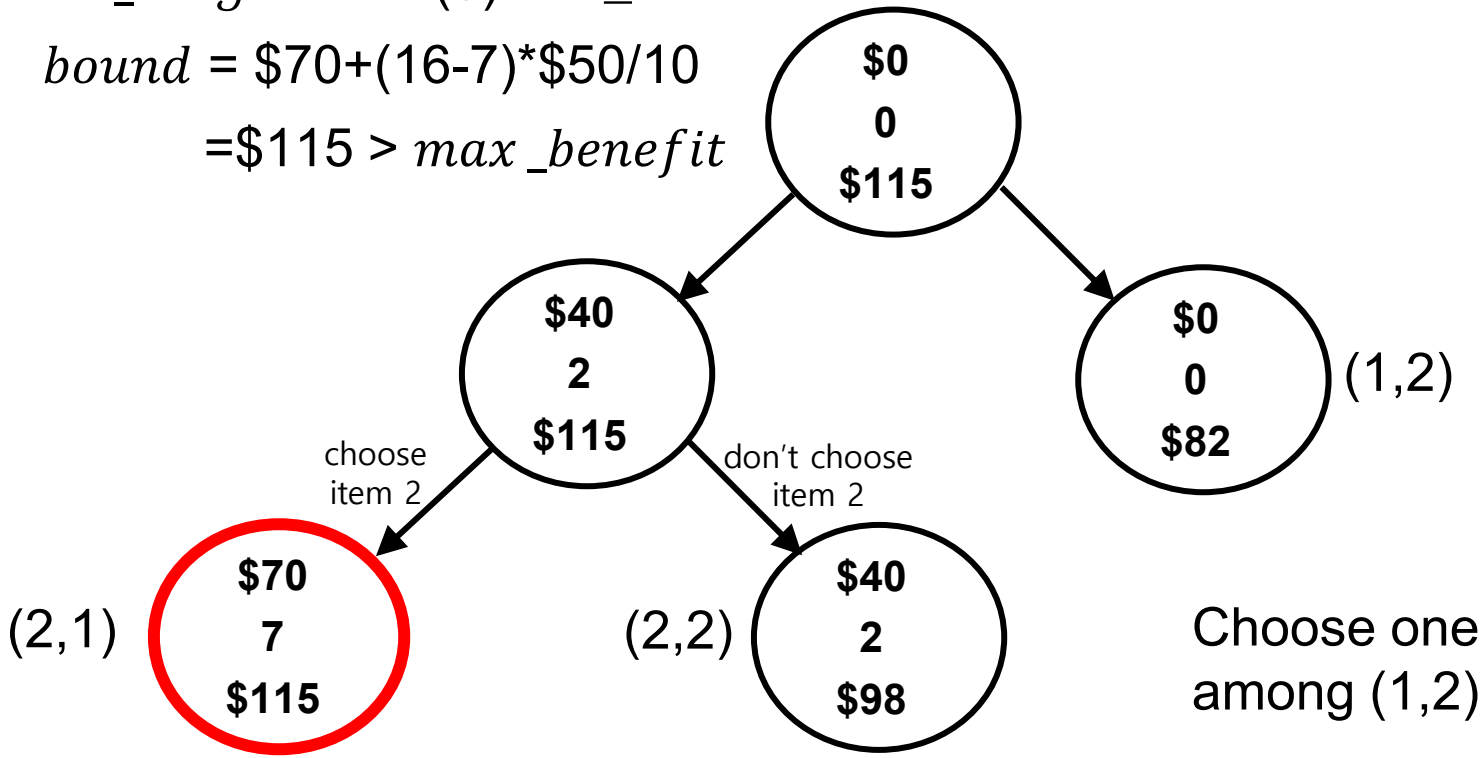
$weight$ = 2+5 = 7 $\leq W = 16$

$max_benefit$ = max(\$40, \$70) = \$70

tot_weight = 7+ (0) = 7 $\leq W = 16$

$bound$ = \$70+(16-7)*\$50/10

=\$115 > $max_benefit$



Choose one node among (1,2) (2,1) (2,2)

$$tot_weight = weight + \sum_{j=i+1}^{k-1} w_j$$

$$bound = benefit + \sum_{j=i+1}^{k-1} b_j + (W - tot_weight) \times \frac{b_k}{w_k}$$

Vertex (3,1)

$benefit$ = \$70+\$50 = \$120

$weight$ = 7+10 = 17 $> W = 16$ (not promising!)

$max_benefit$ is still \$70

Vertex (3,2)

$benefit$ = \$70+0 = \$70

$weight$ = 7 $\leq W = 16$

$max_benefit$ = max(\$70, \$70) = \$70

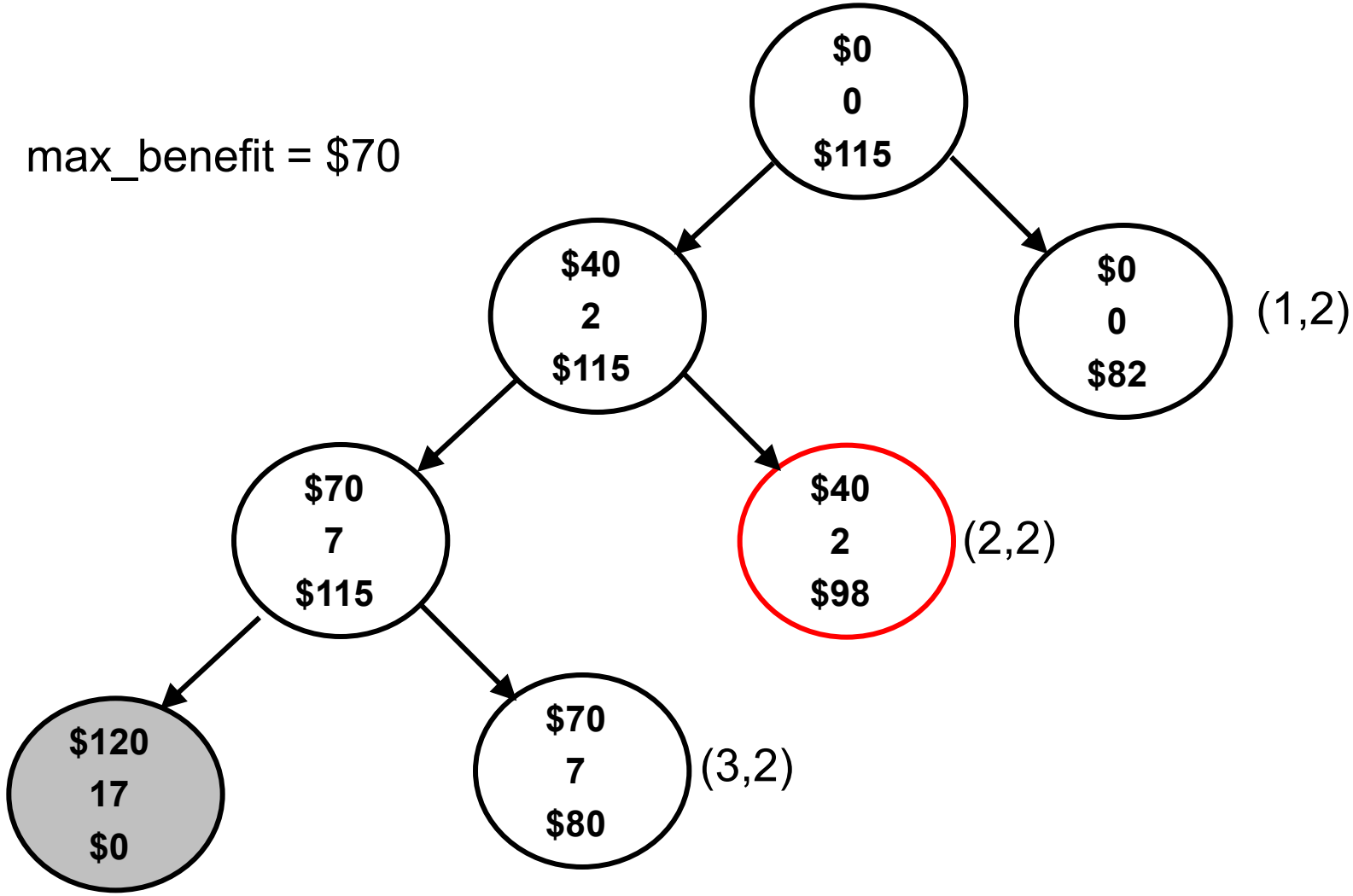
tot_weight = 7+ (5) = 12 $\leq W = 16$

$bound$ = \$70+\$10 = \$80 > max_benefit

b_i	w_i
\$40	2
\$30	5
\$50	10
\$10	5

Branch and Bound

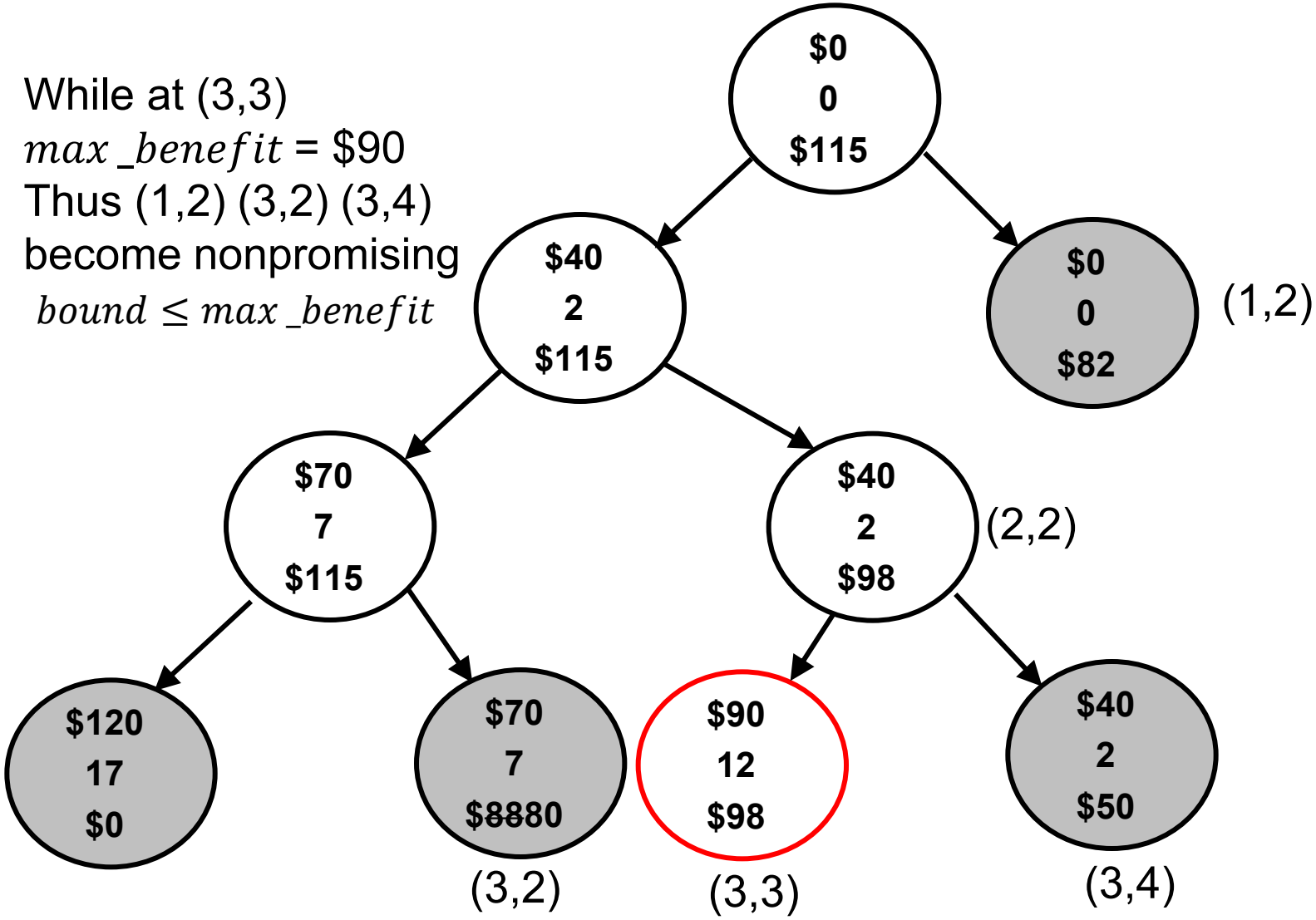
max_benefit = $70



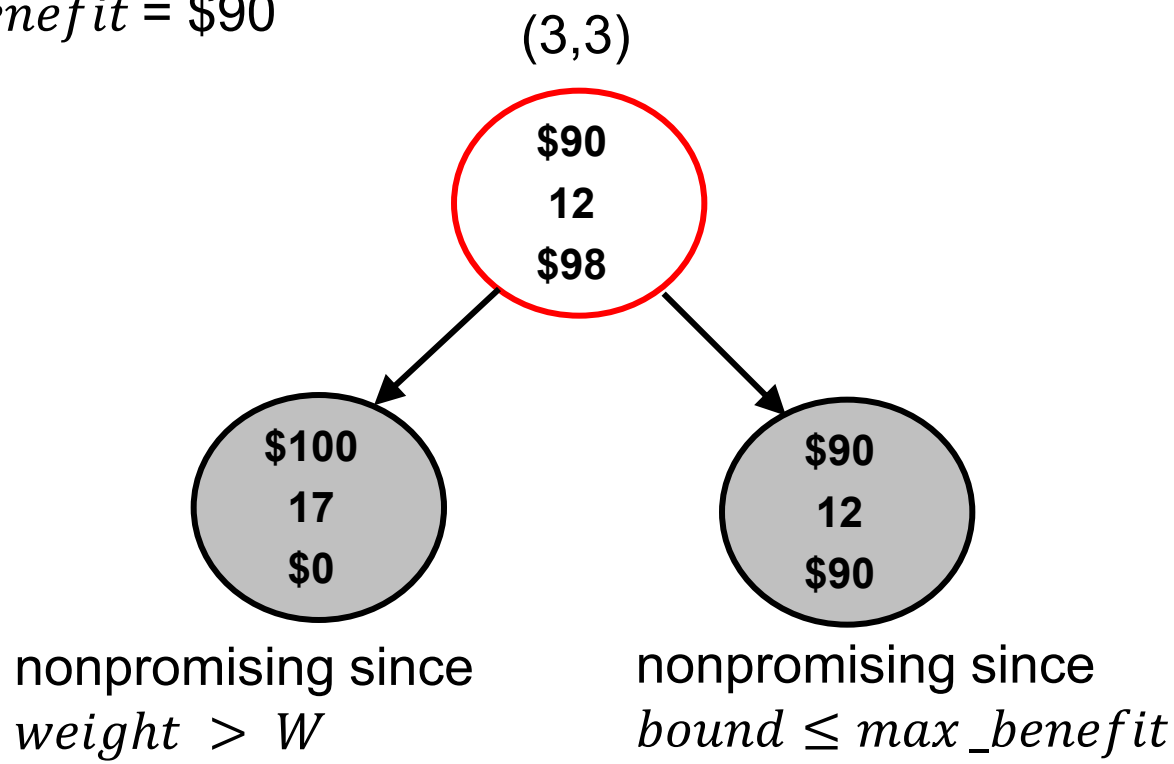
Nonpromising since $weight > W$

Branch and Bound

While at (3,3)
$max_benefit$ = \$90
Thus (1,2) (3,2) (3,4)
become nonpromising
$bound \leq max_benefit$



$max_benefit$ = \$90



Therefore, the vertex (3,3) has $max_benefit$ of \$90.

i.e., choosing item1 and item3 is the best choise.

SUMMARY

Knapsack Problem

- **Greedy algorithm** for knapsack problem
  - ✓ $O(n)$ but unfortunately 0-1 knapsack problem can NOT be solved.
- **Brute force approach** for 0-1 knapsack problem
  - ✓ Running time will be $O(2^n)$
- **Dynamic programming** for 0-1 knapsack problem
  - ✓ $O(n \cdot W)$
- **Branch and bound** for 0-1 knapsack problem
  - ✓ Worst case time complexity is still given by $O(2^n)$
    - in cases where the entire tree has to be explored.
  - ✓ However, its best case time complexity is given by $O(n \log n)$
    - only one path through the tree will have to explored.
    - $O(\log n)$ is needed to sort the items according to w/b.
  - ✓ This method will perform better than the other methods in most scenarios, since multiple branches get eliminated in each iteration.

Thank you!